Generic Decoder-Only Transformer — Build Spec

Config-driven framework: swap pos_encoding / activation / norm without touching the training loop. Target: ~56M params, T4-friendly.

Model config (all fields as one dataclass)

Param	Value	Notes
vocab_size	50257	GPT-2 BPE — reuse existing tokenizer
n_layers	8	transformer blocks
d_model	512	hidden size
n_heads	8	head_dim = d_model / n_heads = 64
head_dim	64	d_model / n_heads
mlp_ratio / d_ff	8/3 -> ~1376	SwiGLU intermediate dim, rounded to mult. of 64
max_seq_len	1024	precompute RoPE cache / ALiBi bias up to this
norm	RMSNorm, pre-norm	applied before attn & before MLP
activation	SwiGLU	gated: swish(xW1) * (xW3), then W2
pos_encoding	{rope, alibi, nope}	config flag, dispatched in Attention
dropout	0.0	unnecessary at this scale/step count
tie_embeddings	True	tie input embed & output lm_head weight
init_std	0.02	normal init, scaled by 1/sqrt(2*n_layers) on output proj

Training config (identical across all variants)

Param	Value	Notes
optimizer	AdamW(0.9, 0.95), wd=0.1	standard LLM defaults
lr_schedule	cosine + linear warmup	warmup_steps=500
peak_lr	3e-4	
batch_size	128 seq (tokens/step const.)	halve batch when seq_len doubles
train_steps	50,000	stop early if val loss plateaus
precision	fp16	T4 = Turing, no native bf16
grad_clip	1.0	global norm
seeds	1 min, 3 for CIs	compute-dependent

Positional encoding module — dispatch table

Mode	Modifies	Formula / mechanism
rope	Q, K (before attn)	rotate pairs: freq_i = base^(-2i/d); apply_rotary(q,k,cos,sin)
alibi	attn scores (after QK^T)	slope_h = 2^(-8h/H); bias[i,j] = -slope_h*|i-j|; scores += bias
nope	nothing	pass-through; causal mask is the only order signal

Attention forward — single dispatch point

```
def forward(self, x, seq_len):
    q, k, v = self.qkv_proj(x)                                  # (B, H, T, Dh)
    if cfg.pos_encoding == "rope": q, k = self.rotary(q, k, seq_len)
    scores = (q @ k.transpose(-2,-1)) / math.sqrt(self.head_dim)
    if cfg.pos_encoding == "alibi": scores = scores + self.alibi_bias[:, :seq_len, :seq_len]
    scores = scores.masked_fill(self.causal_mask[:seq_len,:seq_len]==0, float("-inf"))
    return scores.softmax(dim=-1) @ v
```

Block layout (repeat n_layers times)

```
x = x + Attention(RMSNorm(x))          # pre-norm residual, attention
x = x + SwiGLU_MLP(RMSNorm(x))          # pre-norm residual, MLP
# final: x = RMSNorm(x); logits = x @ tied_embedding.T
```

Param count check: embed(50257*512) + 8 * [attn(4*512*512) + mlp(3*512*1376)] + final_norm ≈ 56M (embeddings tied, so counted once).